PLANNING DOCUMENT

POS v1 Database Design

Schema for a single-branch retail point-of-sale system (Philippines), for SQLite, PostgreSQL, MySQL, Supabase, Google Cloud and AWS

Date	26 October 2026
Scope	Pharmacy, convenience and hardware stores (one shared data model)
Databases	SQLite 3.37+, PostgreSQL 15+, MySQL 8.0.16+; hosted options on Supabase, Google Cloud and AWS (Section 3)
App stack	React + Vite on Electron or Tauri
Files	`sqlite/`, `postgresql/`, `mysql/`: `schema.sql` (tables, views, triggers, seed data) and `example_checkout.sql` (worked examples). `supabase/migrations/`: the PostgreSQL schema plus staff login and access rules.

> **Verify before go-live**
> The discount and VAT rules in this design are a starting point. Check them against current BIR revenue regulations, RA 9994 (Senior Citizens), RA 10754 (PWD) and the current DTI joint administrative order on basic necessities. A POS that issues invoices must also be registered with the BIR.

1. Conventions

What	Stored as	Example
Money	INTEGER centavos	₱112.50 → `11250`
Quantity	INTEGER thousandths (`_milli`)	2 pcs → `2000`; 1.5 m → `1500`
Rates	INTEGER basis points (`_bp`)	12% → `1200`; 20% → `2000`
Date/time	Timestamp, reported in store time (Asia/Manila)	`2026-10-26 14:05:00`
Yes/no	0/1 in SQLite, boolean elsewhere	`is_active`
Choices	TEXT with a CHECK list	`status IN ('pending','paid','voided')`

Whole numbers are used for money and quantities because decimals and floating-point numbers drift (0.1 + 0.2 ≠ 0.3). The app converts to pesos only for display.

These conventions are the same in every version, so the app's calculation code works unchanged whichever database you pick. PostgreSQL and MySQL could store money as `NUMERIC` / `DECIMAL` instead, but then the app would need different code for SQLite.

Every version rejects a value of the wrong type, such as `'112.50'` in a centavo column, instead of silently storing it. SQLite does this through `STRICT` tables, and MySQL through its default strict SQL mode.

> **SQLite only: every connection must start with**
> `PRAGMA foreign_keys = ON;`. SQLite leaves foreign keys off by default, and without this line none of the table links are checked. PostgreSQL and MySQL always check them.

2. Changes from the original plan

Original	v1	Why
Money as String, SubTotal as Double	INTEGER centavos	Strings can't be added; doubles cause rounding errors
Quantity as String	INTEGER thousandths	Math works, and items can be sold by the meter or kilo
Array columns (Category, Emp_role, DiscountType)	`categories` table; single `role`; `discount_types` table	SQLite has no array type; lookups need their own table
One DiscountID per transaction	Discount per line item	An SC gets 20% on medicine but 5% on rice in the same sale
PWD listed as VAT-inclusive	PWD 20% and VAT-exempt, same as SC	RA 10754 gives PWDs the same treatment as seniors
Customer and Membership tables	One `customers` table; walk-ins have no row	Membership duplicated the name fields
ClerkID / EmployeeID (inconsistent)	`clerk_id`, `cashier_id`, `status`	The flow has two people and a slip that exists before payment
Stock as one column	`stock_movements` log + cached `stock_milli`	Every stock change can be traced
Return: one product, no quantity	`returns` + `return_items`; refund or exchange	Multi-item and partial returns, with credit amounts
No supplier, payment or shift tables	`suppliers`, `payments`, `shifts`	Stock-in, split payments, cash drawer and Z-reading
Only SubTotal on each item	Price snapshot plus VAT/discount breakdown	Old receipts don't change when prices change; BIR invoice fields
No SC/PWD ID stored	`beneficiary_type`, `_id_no`, `_name`	BIR requires the ID number and name on the invoice

3. Choosing a database

The same design is provided for three database engines. The cloud options host one of those engines for you, so they use the same files.

Option	What it is	Works offline	Cost	Best for
SQLite	A database inside one file, no server	Yes	Free	One computer per database. The simplest v1
PostgreSQL	Database server you install	Yes, on the store network	Free software; you run the server	Several counters in one store sharing one database
MySQL	Database server you install	Yes, on the store network	Free software; you run the server	Same as PostgreSQL, if the team already knows MySQL
Supabase	Hosted PostgreSQL with login and an API	No	Free plan; paid tiers	Multiple branches or remote reports, without writing a backend
Google Cloud	Cloud SQL: hosted PostgreSQL or MySQL	No	Paid; trial credit for new accounts	Teams already on Google Cloud or Firebase
AWS	RDS / Aurora: hosted PostgreSQL or MySQL	No	Paid; free tier for new accounts	Teams already on AWS, or larger scale

Cloud prices and free-tier rules change often. Check each provider's current pricing page before deciding.

Which files to use

If you run...	Use
SQLite (Electron `better-sqlite3`, Tauri `tauri-plugin-sql`)	`sqlite/`
PostgreSQL on your own server, Google Cloud SQL for PostgreSQL, AlloyDB, AWS RDS for PostgreSQL, Aurora PostgreSQL	`postgresql/`
MySQL on your own server, Google Cloud SQL for MySQL, AWS RDS for MySQL, Aurora MySQL	`mysql/`
Supabase	`supabase/migrations/` (both files, in order)

Online or offline?

A store can't stop selling when the internet drops. That decides most of the architecture:

- **Local only:** SQLite on the counter PC, or PostgreSQL/MySQL on a PC in the store that the counters connect to over the store network. Keeps working without internet. Reports are only available in the store.
- **Cloud only:** Supabase, Google Cloud or AWS. The owner can see every branch from anywhere, but checkout stops when the internet is down.
- **Both:** each counter saves sales to local SQLite and copies them to the cloud database when online. This is the most resilient, but it's also the most work: you have to build the syncing. Each counter also needs its own invoice number series, because two offline counters can't share one counter.

> **Recommendation**
>
> For v1, start with **SQLite**: no server, works offline, and the design is already tested. If you need the cloud, pick **Supabase**. It's PostgreSQL with staff login and access rules already written in `supabase/migrations/`. Choose Google Cloud or AWS when the business already uses them.

4. Entity relationship diagram

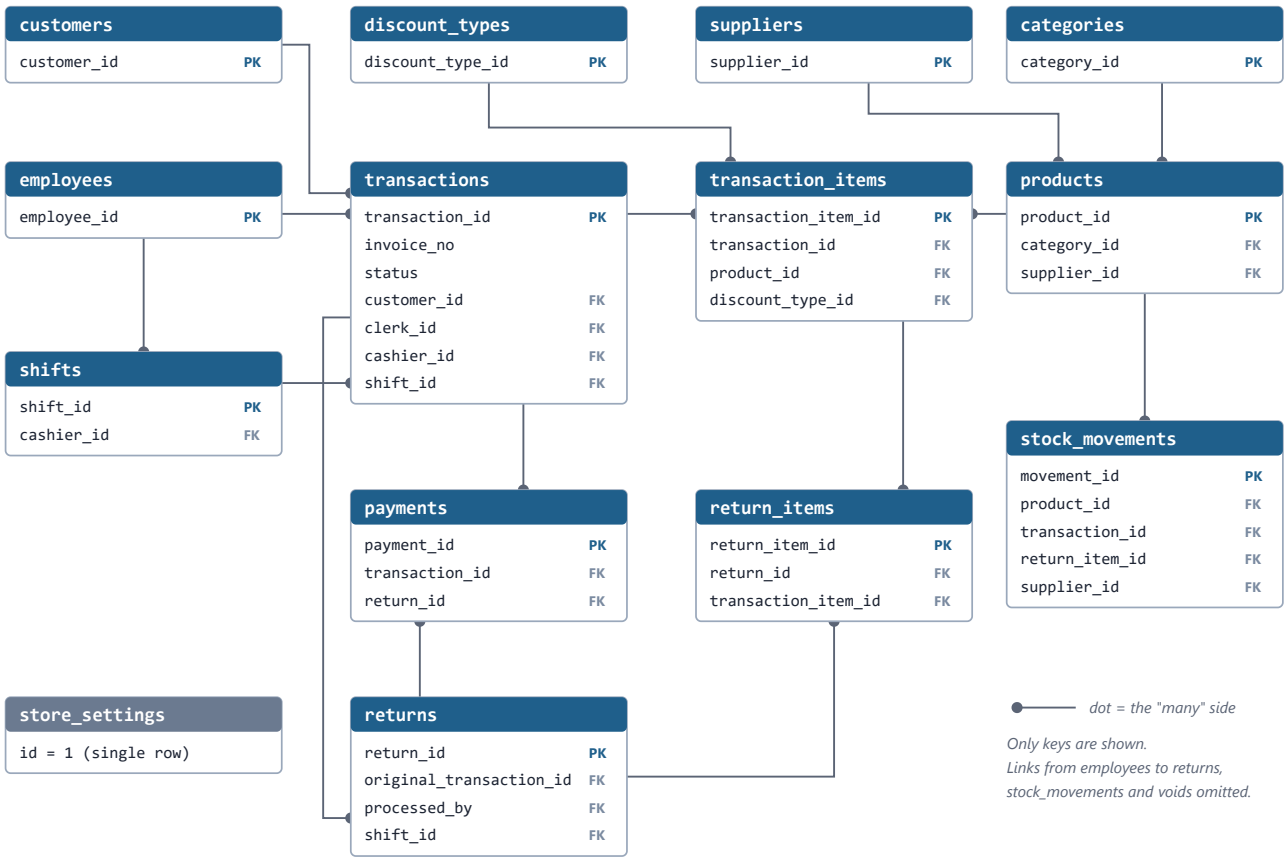


Figure 1. Tables and their links. Each line runs from a foreign key (dot) to the table it points to.

Reading the diagram

- **A sale** is one `transactions` row with many `transaction_items` and one or more `payments`.
- **Each item** points to a `product` and, if discounted, to a `discount_type`.
- **A return** points to the original sale; each `return_item` points to the exact line that was bought.
- **An exchange** is a new sale paid with a `payments` row of method `exchange_credit` that points to the return.
- **Every stock change** is a `stock_movements` row; `products.stock_milli` is their running total.

5. Table reference

Every table also has a whole-number primary key named after the table. `created_at` columns default to the current time. Types below are SQLite's; the other databases use these equivalents:

Kind of value	SQLite	PostgreSQL	MySQL
Primary key	`INTEGER PRIMARY KEY`	`bigint generated always as identity`	`BIGINT AUTO_INCREMENT`
Money (centavos)	`INTEGER`	`bigint`	`BIGINT`
Quantity (thousandths)	`INTEGER`	`integer (bigint for stock)`	`INT (BIGINT for stock)`
Text	`TEXT`	`text`	`VARCHAR(n) / TEXT`
Yes/no (0/1)	`INTEGER 0/1`	`boolean`	`BOOLEAN (TINYINT 0/1)`
Date and time	`TEXT, local time`	`timestamptz (UTC inside)`	`DATETIME, session time zone`
Choice lists	`TEXT + CHECK`	`text + check`	`VARCHAR + CHECK`

PostgreSQL and MySQL also have one extra table, `invoice_counter`: a single row holding the last invoice number, so cashiers paying at the same moment never get the same or a skipped number. `store_settings` in PostgreSQL also has a `time_zone` column (default `Asia/Manila`) used by the daily report.

`store_settings`

Store-wide settings. Exactly one row (`id = 1`).

Column	Type	Notes
`store_name, address, tin`	`TEXT`	Printed on receipts
`is_vat_registered`	`0/1`	If 0, every line is `non_vat` and no VAT is computed (e.g. a sari-sari store under percentage tax)
`vat_rate_bp`	`INTEGER`	Default 1200 (12%)
`return_window_days`	`INTEGER`	Default 7. Past this, a manager must approve the return
`two_step_checkout`	`0/1`	1 = clerk prints an order slip, cashier takes payment. 0 = one person does both

`employees`

Staff who log in. Permissions are nested: a cashier can do clerk work; a manager can do everything.

Column	Type	Notes
`full_name`	`TEXT`	
`role`	`TEXT`	`clerk` | `cashier` | `manager`
`pin_hash`	`TEXT`	bcrypt or argon2 hash of the login PIN, never the PIN itself
`is_active`	`0/1`	Deactivate instead of deleting, so old sales keep their staff link

`categories` · `suppliers`

Lookup tables. `categories.name` is unique. Suppliers hold name, contact person, phone, email, address and `is_active`.

products

The catalog.

Column	Type	Notes
sku	TEXT	Unique, required
barcode	TEXT	Unique; empty if the item has none. Scanners type this into the search field
name	TEXT	
category_id FK	INTEGER	Required
supplier_id FK	INTEGER	Optional; main supplier
unit	TEXT	`pc`, `box`, `m`, `kg`, `L` …
allow_fractional	0/1	1 = can sell 1.5 m or 0.25 kg. 0 = whole units only (enforced)
price_cents	INTEGER	Shelf price, VAT-inclusive if the store is VAT-registered
cost_cents	INTEGER	For margin reports
vat_status	TEXT	`vatable` | `exempt` | `zero_rated`
discount_class	TEXT	`sc_pwd` (20% + VAT-exempt for SC/PWD, e.g. medicine) | `basic_necessity` (5% for SC) | `none`
stock_milli	INTEGER	On hand. Updated automatically from `stock_movements`; never edit directly
reorder_milli	INTEGER	Shows in `v_low_stock` at or below this level
is_active, updated_at	0/1, TEXT	

stock_movements

One row per stock change. Positive adds stock, negative removes it.

Column	Type	Notes
product_id FK	INTEGER	
qty_milli	INTEGER	Signed, never 0
reason	TEXT	`stock_in` | `sale` | `sale_void` | `return` | `adjustment`
transaction_id, return_item_id, supplier_id FK	INTEGER	Whichever applies
unit_cost_cents	INTEGER	For `stock_in`
employee_id FK, note	INTEGER, TEXT	Who did it, and why (needed for adjustments)

customers

Registered customers and members. Walk-ins get no row.

Column	Type	Notes
full_name, phone, email	TEXT	Name required
is_member, member_since	0/1, TEXT	Replaces the separate Membership table

`discount_types`

Discount rules, applied per line item. Seeded with `SC_20`, `PWD_20` and `SC_BN_5`.

Column	Type	Notes
`code, name`	`TEXT`	Code is unique
`rate_bp`	`INTEGER`	0 to 10000
`vat_exempt`	`0/1`	1 = remove VAT first, then discount (SC/PWD 20%)
`beneficiary_type`	`TEXT`	`SC` | `PWD`; empty = anyone (store promo)
`product_class`	`TEXT`	Matches `products.discount_class`; empty = any product
`weekly_cap_cents`	`INTEGER`	Purchase cap per beneficiary per week. Left empty for `SC_BN_5`; set it from the current DTI order
`valid_from, valid_to, is_active`	`TEXT, 0/1`	For time-limited promos

`shifts`

A cashier's drawer session, for cash counts and end-of-day reports.

Column	Type	Notes
`cashier_id` FK	`INTEGER`	
`opened_at, closed_at`	`TEXT`	
`opening_cash_cents`	`INTEGER`	Starting float
`counted_cash_cents`	`INTEGER`	Counted at close
`expected_cash_cents`	`INTEGER`	Opening + cash sales − cash refunds. Over/short = counted − expected

`transactions`

One sale, from order slip to paid invoice. Never deleted, only voided.

Column	Type	Notes
`invoice_no`	`INTEGER`	Assigned automatically when paid. Sequential and never reused
`status`	`TEXT`	`pending` (slip) → `paid` → `voided`
`customer_id` FK	`INTEGER`	Empty for walk-ins
`clerk_id` FK	`INTEGER`	Who built the order (required)
`cashier_id` FK	`INTEGER`	Who took payment (required once paid)
`shift_id` FK	`INTEGER`	
`beneficiary_type`, `beneficiary_id_no`, `beneficiary_name`	`TEXT`	SC/PWD details for the invoice. Type and ID number go together; name is required with them
`gross`, `vat_removed`, `discount`, `total`	`INTEGER`	`_cents`. Frozen from the line items at payment, so a printed invoice never changes
`vatable_sales`, `vat`, `vat_exempt_sales`, `zero_rated_sales`	`INTEGER`	`_cents`. The VAT breakdown printed on the invoice
`paid_at`	`TEXT`	Set automatically
`voided_at`, `voided_by` FK, `void_reason`	`TEXT`, `INT`, `TEXT`	All three required to void

`transaction_items`

One line on the order. All amounts are computed by the app (Section 7) and checked by the database.

Column	Type	Notes
`transaction_id`, `product_id` FK	`INTEGER`	
`qty_milli`	`INTEGER`	More than 0
`unit_price_cents`	`INTEGER`	Copy of the product price at the time of sale
`tax_treatment`	`TEXT`	This line's actual treatment: `vatable` | `exempt` (incl. SC/PWD exemption) | `zero_rated` | `non_vat`
`discount_type_id` FK	`INTEGER`	Required if there is a discount
`gross_cents`	`INTEGER`	Unit price × quantity
`vat_removed_cents`	`INTEGER`	VAT taken off for an SC/PWD exemption
`discount_cents`	`INTEGER`	
`net_cents`	`INTEGER`	What the customer pays. Must equal gross − vat_removed − discount
`vat_cents`	`INTEGER`	VAT contained in net (vatable lines only)

payments

Money received for a sale. Several rows = split payment. Total must equal the amount due.

Column	Type	Notes
transaction_id FK	INTEGER	
method	TEXT	cash | ewallet | card | exchange_credit
amount_cents	INTEGER	Amount applied to the bill
tendered_cents	INTEGER	Cash handed over. Change = tendered − amount
reference_no	TEXT	E-wallet or card reference
return_id FK	INTEGER	Required for, and only for, exchange_credit

returns

A return against one paid sale.

Column	Type	Notes
original_transaction_id FK	INTEGER	The sale being returned
processed_by, approved_by FK	INTEGER	Approver = manager, when past the return window
shift_id FK	INTEGER	For the cash drawer count
resolution	TEXT	refund | exchange
reason	TEXT	Required
cash_refund_cents	INTEGER	Cash given back. For an exchange, only the leftover credit

return_items

Each line being returned.

Column	Type	Notes
return_id, transaction_item_id FK	INTEGER	The exact original line
qty_milli	INTEGER	Cannot exceed what is left of the original line
credit_cents	INTEGER	What the customer paid for these units: net × returned qty ÷ sold qty
item_condition	TEXT	resellable (back to stock) | damaged (not restocked)

Views

View	Use
v_transaction_totals	Live totals for any transaction, computed from its lines. Show this on a pending slip. The payment step copies it into transactions.
v_low_stock	Active products at or below their reorder level.
v_daily_sales	Per-day invoice range, paid and voided counts, gross, discounts, net sales and VAT breakdown. Basis for the Z-reading.

6. How a sale works

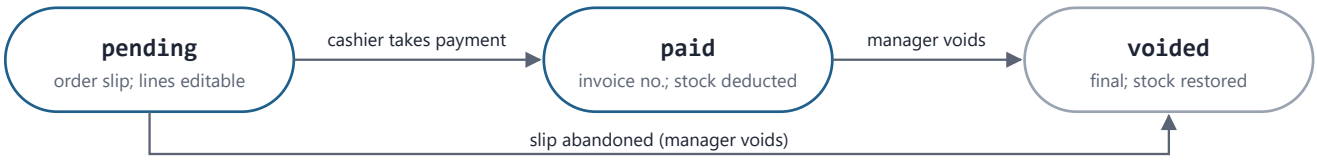


Figure 2. Transaction status. No other changes are allowed.

Steps

1. **Clerk builds the order.** Insert a `transactions` row (status `pending`, `clerk_id`). If the customer is SC/PWD, check the ID now and fill in the beneficiary fields, before the slip prints.
2. **Add lines.** For each scanned item, insert a `transaction_items` row with the amounts computed as in Section 7. Lines can be changed freely while pending.
3. **Print the order slip** from `v_transaction_totals`.
4. **Cashier takes payment.** Insert one or more `payments` rows that add up to the amount due.
5. **Mark it paid:** `UPDATE transactions SET status = 'paid', cashier_id = ?, shift_id = ?`. The database then:
   - checks there are items and the payments match the amount due,
   - assigns the next invoice number and the paid time (PostgreSQL and MySQL take it from `invoice_counter`),
   - copies the totals and VAT breakdown onto the transaction,
   - deducts stock through `stock_movements`.
6. **Print the invoice.** From here the lines and payments are locked.

> **Wrap each step in a database transaction**
> `BEGIN … COMMIT` makes the step all-or-nothing. If the app crashes halfway through payment, nothing is half-saved. In single-step stores (`two_step_checkout = 0`), the same person does all steps and the slip is skipped.

7. SC / PWD discounts and VAT

Discounts are applied to each line, not the whole sale. Each line gets at most one discount; the app picks the best one the customer qualifies for. SC/PWD discounts do not stack with store promos.

Code	Who	Which products	Rate	VAT
`SC_20`	Senior Citizen	`discount_class = 'sc_pwd'` (e.g. medicine)	20%	Exempt
`PWD_20`	PWD	`discount_class = 'sc_pwd'`	20%	Exempt
`SC_BN_5`	Senior Citizen	`discount_class = 'basic_necessity'`	5%	Unchanged; weekly cap

Line calculation (done by the app)

```
gross        = unit_price × qty
if the discount is VAT-exempt (SC/PWD 20%):
    base = round(gross / 1.12)     vat_removed = gross − base
else:
    base = gross                   vat_removed = 0
discount     = round(base × rate)
net          = base − discount
vat          = (tax_treatment = 'vatable') ? net − round(net / 1.12) : 0
```

Round half up to the centavo, per line. If the store is not VAT-registered, skip the VAT step: the 20% is taken straight off the price and every line is `non_vat`.

Worked example: Senior Citizen sale

Item	Qty	Gross	VAT removed	Discount	Net
Paracetamol 500mg, box (₱112.00, `SC_20`)	2	224.00	24.00	40.00	160.00
Rice, per kg (₱56.00, VAT-exempt item, `SC_BN_5`)	2 kg	112.00	0.00	5.60	106.40
Total		**336.00**	**24.00**	**45.60**	**266.40**

One box of paracetamol: ₱112.00 ÷ 1.12 = ₱100.00 without VAT, less 20% = **₱80.00**.

The invoice shows VAT-exempt sales *before* the SC discount (₱312.00 here), then the discount on its own line. This is the usual layout, but confirm it with your accountant or BIR.

For the 5% basic-necessities discount, the app must add up this beneficiary's purchases over the past 7 days (by `beneficiary_id_no`) and stop applying it once the weekly cap is reached.

```
        MY STORE
   SALES INVOICE No. 000001

Paracetamol 500mg  2 x 112.00
                        224.00
Rice (kg)       2.000 x 56.00
                        112.00
-----------------------------
Total                   336.00
Less: VAT                24.00
Less: SC discount        45.60
AMOUNT DUE              266.40

Cash                    300.00
Change                   33.60
-----------------------------
VATable sales             0.00
VAT amount                0.00
VAT-exempt sales        312.00
Zero-rated sales          0.00

SC ID No.: SC-12345
Name:      Juan Dela Cruz
Signature: _________________
```

8. Returns and exchanges

1. **Find the original sale** by invoice number from the receipt. It must be `paid`.
2. **Check the return window** (`return_window_days`). Past it, a manager approves and is recorded in `approved_by`.
3. **Create the return** with a reason, then a `return_items` row for each line, with the quantity, condition and credit. Credit is what the customer actually paid: an SC who paid ₱80.00 for a ₱112.00 box gets ₱80.00 back, not ₱112.00.
4. **Resellable items go back into stock** automatically. Damaged items do not.
5. **Refund:** set `cash_refund_cents` to the total credit.
6. **Exchange:** ring up the new items as a new sale. Pay it with a `payments` row of method `exchange_credit` pointing to the return.
   - If the new items cost more, the customer pays the difference with another payment.
   - If they cost less, refund the leftover in `cash_refund_cents`.

> **Void or return?**
> Void a sale only when it is cancelled entirely, before the customer leaves. Once anything from a sale has been returned, it can no longer be voided.

9. Rules the database enforces

These are built into every `schema.sql` as constraints and triggers, so they hold even if the app has a bug. They were tested on SQLite and PostgreSQL. The MySQL version has the same rules but has not been run yet (Section 11).

Rule	What happens
Wrong data type	Rejected (e.g. text or `112.50` in a centavo column)
Line amounts don't add up	Rejected: net must equal gross − VAT removed − discount
Whole-unit product sold in a fraction	Rejected: "this product is sold in whole units only"
Paying an empty order, or payments ≠ amount due	Rejected
Marking paid	Invoice number, paid time and totals set; stock deducted
Editing lines or adding payments after payment	Rejected: "transaction is no longer pending"
Deleting a transaction	Rejected: void it instead
Void without who, when and why	Rejected
Voiding a paid sale	Stock put back. Rejected if the sale has returns
Changing a voided transaction, or going back to pending	Rejected
SC/PWD without ID number or name	Rejected
Returning an item from a different sale, or more than was bought	Rejected
Exchange credit plus cash refund more than the return is worth	Rejected

Any stock movement	`products.stock_milli` updated to match
Changing or deleting a stock movement	Rejected: add an adjustment instead
Creating a sale that is already marked paid	Rejected: every sale starts as a pending order
Editing anything on a paid invoice except voiding it	PostgreSQL and MySQL: rejected. SQLite: lines and payments are locked, but the app must not change the header

10. Setting up each database

SQLite

Files: `sqlite/schema.sql`, `sqlite/example_checkout.sql`.

- When the app first starts, create the database file and run `schema.sql` once. With Electron and `better-sqlite3`: `db.exec(fs.readFileSync('schema.sql', 'utf8'))`.
- Run `PRAGMA foreign_keys = ON` on every connection, and `PRAGMA journal_mode = WAL` once.
- Keep the database file in the app's user-data folder, not the install folder, and back it up daily (a copy of the file is a full backup).
- Only one computer should write to the file. Don't share it over a network drive; use a server database for that.

PostgreSQL

Files: `postgresql/schema.sql`, `postgresql/example_checkout.sql`. Needs PostgreSQL 15 or newer.

```
createdb pos
psql -d pos -f postgresql/schema.sql
psql -d pos -f postgresql/example_checkout.sql   # optional: sample data
```

- Connect from the app with the `pg` package (Node) or `tauri-plugin-sql` (Tauri).
- Times are stored as `timestamptz` (UTC inside). The daily report converts them using `store_settings.time_zone`.
- For a store network, install PostgreSQL on one PC, allow the counters' addresses in `pg_hba.conf`, and give the app its own database user, not `postgres`.

MySQL

Files: `mysql/schema.sql`, `mysql/example_checkout.sql`. Needs MySQL 8.0.16 or newer (older versions ignore CHECK rules).

```
mysql -u root -p -e "CREATE DATABASE pos CHARACTER SET utf8mb4"
mysql -u root -p pos < mysql/schema.sql
mysql -u root -p pos < mysql/example_checkout.sql   # optional: sample data
```

- Load it with the `mysql` command-line client. The `DELIMITER` lines in the file are client commands; some GUI tools need them removed.
- Set the time zone to store time on the server, or run `SET time_zone = '+08:00'` on every connection.
- Keep the default strict SQL mode so bad values are rejected.
- Connect from the app with the `mysql2` package.
- XAMPP ships MariaDB, not MySQL. It is very similar, but this schema has not been tried on it.

Supabase

Files: `supabase/migrations/20261026000000_pos_schema.sql` (same as the PostgreSQL schema) and `…000100_auth_and_rls.sql` (staff login and access rules).

1. Create a project in the Southeast Asia (Singapore) region, the closest to the Philippines.
2. Apply both files in order: `supabase db push` with the Supabase CLI, or paste each into the SQL Editor.
3. For each staff member, create a user under Authentication, then link it: `update employees set auth_user_id = '…' where employee_id = …`.
4. In the app, log in with `supabase.auth.signInWithPassword()` and use the normal client. The access rules apply automatically.

Role	Allowed (everything else is refused by the database)
Clerk	Read everything except PIN hashes. Start orders under their own name; add and edit lines on pending orders; register customers.
Cashier	Clerk permissions, plus: take payments, mark orders paid as themselves, open and close their own shifts, process returns.
Manager	Everything, plus: void sales, edit products, prices, discounts and staff, record deliveries and stock adjustments, approve late returns.
Not logged in	Nothing.

- Never put the `service_role` key in the POS app; it bypasses every rule. Only the public (anon) key belongs there.
- Staff can't read `pin_hash`, so query employees with named columns, not `select *`.
- On the free plan, inactive projects can be paused. Check the current plan limits before relying on it for a live store.

Google Cloud

- **Use Cloud SQL** for PostgreSQL (with `postgresql/`) or Cloud SQL for MySQL (with `mysql/`). AlloyDB also takes the PostgreSQL files.
- Pick region `asia-southeast1` (Singapore). For MySQL, set the `default_time_zone` flag to `+08:00`.
- Don't let the counter PCs connect to the database directly over the internet. Put a small backend in between, such as Cloud Run, connected through the Cloud SQL connector. The desktop app talks to that backend, which checks who the user is.
- **Firebase / Firestore is not a good fit.** It's a document database with no table links, CHECK rules or triggers, so every rule in Section 9 would have to be rewritten as Cloud Functions. Sales and stock data is relational. If you want Firebase, use Firebase Authentication with Cloud SQL, or Firebase Data Connect, which runs on Cloud SQL for PostgreSQL.

AWS

- **Use RDS for PostgreSQL or Aurora PostgreSQL** (with `postgresql/`), or RDS for MySQL or Aurora MySQL (with `mysql/`).
- Pick region `ap-southeast-1` (Singapore). For MySQL, set `time_zone` in a DB parameter group.
- Keep the database private (not publicly accessible). Put a backend in between, such as Lambda + API Gateway or a small server on ECS/EC2, and keep credentials in Secrets Manager.
- Turn on automated backups and choose a retention period.
- **DynamoDB is not a good fit**, for the same reasons as Firestore.

11. What the app must do, and open items

The app is responsible for

- SQLite: running `PRAGMA foreign_keys = ON` on every connection.
- Cloud databases: going through a backend or Supabase login, never embedding database admin credentials in the desktop app.
- Computing each line's VAT and discount (Section 7) before saving it.
- Converting pesos to centavos. The database can't tell `112` meant as ₱112.00 from 112 centavos.
- Choosing the best discount for each line, and enforcing the weekly cap on basic necessities.
- Checking the return window, and asking for a manager PIN for voids and late returns.
- Checking role permissions (clerk / cashier / manager).
- Hashing PINs (bcrypt or argon2).
- Computing `expected_cash_cents` when a shift closes.

Open items

Item	Note
Weekly cap amount for `SC_BN_5`	Left empty. Set it from the current DTI joint administrative order.
5% basic necessities for PWD	Not seeded. Check whether current rules extend it to PWDs.
Refunds in the Z-reading	`v_daily_sales` shows sales only. Refunds live in `returns`; add them to the end-of-day report.
Locking totals on paid invoices (SQLite)	In SQLite, lines and payments are locked, but the header totals of a paid sale can still be updated. PostgreSQL and MySQL lock them.
Invoice layout and BIR registration	Confirm required invoice fields and the VAT-exempt sales presentation with BIR or an accountant.
Offline sync	If you combine local SQLite with a cloud database, the syncing and per-counter invoice series still need to be designed.
Testing	**SQLite** (3.45) and **PostgreSQL** (18): walkthrough and all rules tested. **Supabase** files: tested on PostgreSQL 18 with a stand-in for Supabase login, checking every role's permissions; not yet run on a real Supabase project. **MySQL**: written but **not yet run**, because no MySQL server was available. None of them has been run through Electron or Tauri yet.